

Optimal Control of an Underactuated Rotary Double Pendulum: A Comparative Study of LQR, iLQR, and MPC

Sara Al Sowaimel

Tianqin Fu

Alexander Wegener

April 24, 2026

1 Introduction & Motivation

Underactuated mechanical systems with nonlinear rigid body dynamics appear across a broad range of engineering applications — from legged robots, whose center-of-mass dynamics reduce to an underactuated inverted pendulum actuated only at the contact foot [1], to serial manipulators with intentionally unactuated distal links to reduce weight and energy consumption [2]. In such systems, the number of control inputs is smaller than the number of degrees of freedom, and stabilization must be achieved indirectly through state coupling, often governed by nonlinear dynamics and gravity.

The rotary double pendulum captures this structure in a minimal and analytically tractable form. With two degrees of freedom and a single actuator, it provides a representative example of a nonlinear underactuated system while remaining simple enough to allow systematic analysis and comparison [3]. At the same time, it retains key characteristics that make control challenging, including an unstable upright equilibrium, strong nonlinear coupling between links, and sensitivity to disturbances.

The prevalence of such systems has led to the development of a wide range of control approaches, each reflecting different modeling assumptions and computational trade-offs. Linear Quadratic Regulation (LQR) provides an efficient solution based on local linearization, but is only valid near the operating point [3]. Iterative LQR (iLQR) extends this idea to nonlinear dynamics through trajectory-based optimization, improving performance over a larger region of the state space [4]. Model Predictive Control (MPC) further incorporates explicit handling of system constraints, at the cost of solving an optimization problem at each control step [5].

These methods are well studied individually, and prior work has examined comparisons between linear quadratic control, trajectory optimization methods, and model predictive control in various theoretical and applied contexts [6–8]. However, their relative performance under consistent conditions — particularly in terms of robustness to disturbances, constraint handling, and computational feasibility — remains an important practical question.

In this work, we perform a systematic comparison of LQR, iLQR, and nonlinear MPC on a shared rotary double pendulum system. All controllers are evaluated within a unified simulation framework under identical test conditions, including sensor noise, actuator limits, model mismatch, and external disturbances. In addition, controller performance is analyzed not only in terms of control quality, but also computational requirements, including deployment on resource-constrained hardware.

The goal of this study is to characterize the trade-offs between performance, robustness, and computational cost, and to identify the regimes in which each control strategy is most effective.

2 Problem Formulation

2.1 System Description

We consider a rotary double pendulum with two degrees of freedom and a single actuated joint. The system state is defined as

$$x = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top \in \mathbb{R}^4,$$

where θ_1 denotes the actuated base joint angle and θ_2 the passive joint angle. The control input is the torque applied at the base joint,

$$u = \tau \in \mathbb{R}.$$

2.2 Dynamics

The system evolves according to nonlinear dynamics of the form

$$\dot{x} = f(x, u),$$

derived from the Euler–Lagrange equations of motion for a two-link rigid body system. The explicit form of $f(\cdot)$ is given in Section 3.

2.3 Control Objective

The goal is to stabilize the system at the unstable upright equilibrium

$$x^* = [\pi/2 \quad 0 \quad 0 \quad 0]^\top,$$

starting from an initial state near the downward configuration, and to maintain stability in the presence of disturbances.

This objective is formulated as an optimal control problem that penalizes deviations from the target state as well as control effort. A general quadratic cost of the form

$$J = \int_0^T (x - x^*)^\top Q (x - x^*) + u^\top R u \, dt$$

is used, where $Q \succeq 0$ and $R \succ 0$ are weighting matrices.

2.4 Constraints

The control input is subject to actuator limitations of the form

$$|u(t)| \leq u_{\max},$$

reflecting physical torque limits of the system. These constraints are explicitly incorporated in the MPC formulation, while LQR and iLQR handle them implicitly through saturation.

2.5 Evaluation Criteria

All controllers are evaluated with respect to their ability to stabilize the system and maintain performance under perturbations. Performance is assessed in terms of tracking accuracy, control effort, robustness to disturbances and model mismatch, and computational cost. This formulation is shared across all controllers, with differences arising in how the control problem is approximated and solved.

3 System Modeling

3.1 Mechanical Configuration

The system is a rotary double pendulum mounted on a fixed vertical axis, illustrated in 1. The base joint (joint 1) rotates about the vertical axis and is the single actuated degree of freedom, driven by a torque input τ . Link 1 connects the base to link 2, which is attached via a passive joint (joint 2) and moves in the plane defined by link 1.

The system has two degrees of freedom and a single control input, resulting in one degree of underactuation. The generalized coordinates are $\mathbf{q} = [\theta_1, \theta_2]^\top$, with corresponding state $\mathbf{x} = [\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top \in \mathbb{R}^4$.

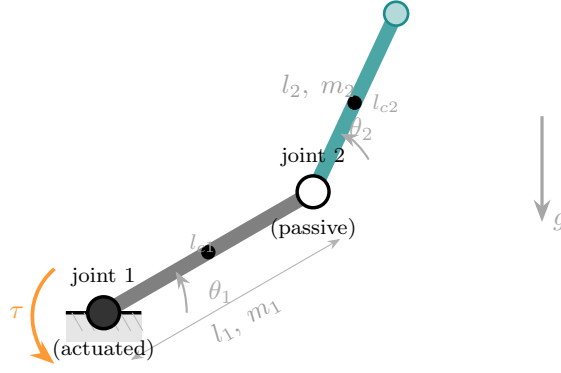


Figure 1: Rotary double pendulum. Joint 1 is actuated by torque τ , while joint 2 is passive.

3.2 Dynamics

The system dynamics follow the Euler–Lagrange formulation for a two-link rigid body system,

$$M(\mathbf{q}) \ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + g(\mathbf{q}) = B \tau, \quad (1)$$

where $M(\mathbf{q})$ is the configuration-dependent mass matrix, $C(\mathbf{q}, \dot{\mathbf{q}})$ captures Coriolis and centripetal effects, and $g(\mathbf{q})$ represents gravitational torques. The input matrix $B = [1, 0]^\top$ reflects that actuation is applied only at the first joint.

For control design, the dynamics are expressed in state-space form as

$$\dot{\mathbf{x}} = f(\mathbf{x}, \tau), \quad (2)$$

consistent with the formulation in Section 2.

A linearization of 1 around the upright equilibrium $\mathbf{x}^* = [\pi/2, 0, 0, 0]^\top$ yields a linear time-invariant approximation

$$\dot{\mathbf{x}} = A\mathbf{x} + B_u \tau, \quad (3)$$

where A and B_u are obtained from the Jacobians of the nonlinear dynamics. This linearized model is used for LQR design and for constructing terminal costs in MPC.

4 Control Methods

The control objective is addressed using three controllers of increasing capability. Each relaxes a limiting assumption of its predecessor: LQR assumes linear dynamics and imposes no constraints; iLQR handles the full nonlinear dynamics at moderate additional cost; finally, MPC incorporates explicit constraint handling at the expense of solving an optimization problem at every control step. Each controller is described in more detail in the following subsections, with emphasis on the specific formulation and implementation used in this work.

4.1 Linear Quadratic Regulator (LQR)

4.1.1 Model

The LQR controller is based on a local linearization of the nonlinear dynamics around the upright equilibrium x^* . A first-order approximation of the system dynamics is used to obtain a linear time-invariant model, which serves as the basis for controller design. This approximation is only valid in a neighborhood of the equilibrium, and therefore restricts the applicability of LQR to local stabilization. The linearization is performed numerically using finite-difference perturbations of the nonlinear dynamics obtained from the MuJoCo simulation model.

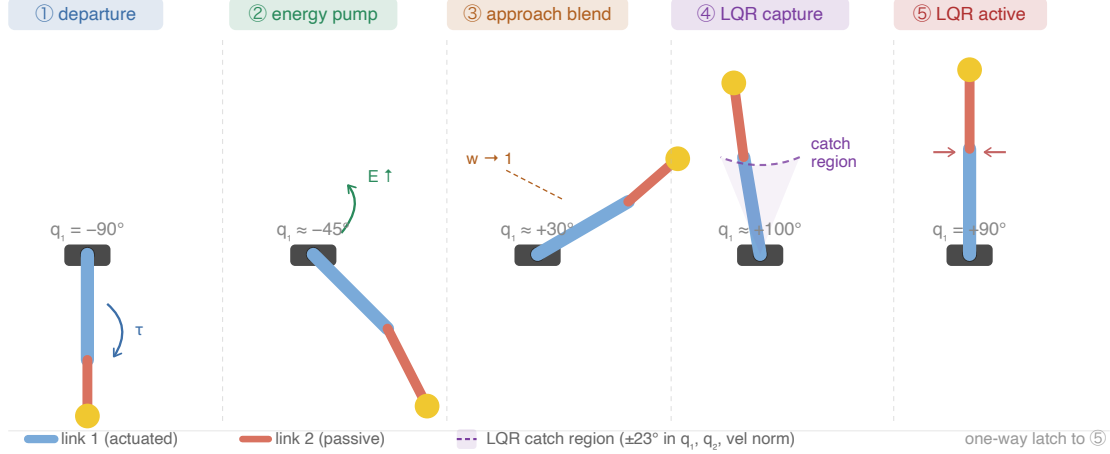


Figure 2: Control phases of the LQR swing-up and stabilisation sequence. Link 1 (blue) is actuated; link 2 (orange) is passive. The shaded wedge in phase 4 marks the four-condition catch region; entry latches the controller permanently into LQR mode (phase 5).

4.1.2 Control Formulation

Within this local model, a discrete-time infinite-horizon LQR problem is formulated, consistent with the general optimal control objective introduced in Section 2. The controller minimizes

$$J = \sum_{k=0}^{\infty} \left[(x_k - x^*)^\top Q (x_k - x^*) + u_k^\top R u_k \right],$$

with $Q \succeq 0$ and $R \succ 0$. Solving the DARE during controller initialization yields a constant feedback gain K , leading in the control law

$$u_k = -K(x_k - x^*).$$

4.1.3 Implementation

Since the LQR gain is only valid near the upright equilibrium, a three-phase swing-up precedes stabilization, illustrated in Fig. 2.

At startup a short torque pulse (①) breaks the symmetry of the hanging equilibrium. A passivity-based energy-shaping law (②) then drives total mechanical energy toward the upright value via

$$\tau \propto -\tanh\left(\frac{\Delta E}{\sigma}\right) \operatorname{sgn}(\dot{\theta}_1 \cos \theta_1), \quad (4)$$

with a small θ_2 damping term to suppress passive-joint oscillation. As the system approaches the top, a smooth blend weight $w \in [0, 1]$ transitions control authority from energy shaping to a PD law centred on the equilibrium (③), avoiding a hard switch that would excite the passive joint.

The transition to LQR triggers when $|\tilde{\theta}_1| < 0.40$ rad, $|\tilde{\theta}_2| < 0.45$ rad, $\|\dot{\tilde{q}}\| < 3.0$ rad/s, and $|\Delta E| < 0.60$ J are simultaneously satisfied. The velocity and energy conditions prevent catches with residual kinetic energy the linear gain cannot damp. The switch is a one-way latch (④→⑤). The gain K is computed once at initialisation from the DARE on a ZOH-discretised linearisation at $\Delta t = 2.5$ ms, with $Q = \operatorname{diag}(30, 300, 5, 40)$ and $R = 1$.

4.2 Iterative Linear Quadratic Regulator (iLQR)

4.2.1 Model

The iLQR controller operates on the full nonlinear dynamics from Section 3, discretized at $\Delta t = 0.02$ s by fourth-order Runge–Kutta integration: $x_{k+1} = f(x_k, \tau_k)$. Unlike LQR, the dynamics are re-linearized about

the current candidate trajectory at every iteration, enabling the controller to plan the swing-up maneuver directly rather than relying on an external energy-shaping phase.

4.2.2 Control Formulation

Over a planning horizon of N steps, iLQR minimizes

$$\min_{\tau_{0:N-1}} \sum_{k=0}^{N-1} \left(\frac{1}{2} \tilde{x}_k^\top Q \tilde{x}_k + \frac{1}{2} R \tau_k^2 \right) + \frac{1}{2} \tilde{x}_N^\top P \tilde{x}_N, \quad (5)$$

subject to $x_{k+1} = f(x_k, \tau_k)$, with $\tilde{x}_k = x_k - x^*$. Angles are wrapped to $[-\pi, \pi]$ to keep the cost smooth across the boundary. The algorithm follows the Gauss–Newton variant of Differential Dynamic Programming. Each iteration runs a backward pass returning feedforward–feedback gains

$$k_k = -(Q_{uu} + \mu)^{-1} Q_u, \quad K_k = -(Q_{uu} + \mu)^{-1} Q_{ux}, \quad (6)$$

followed by a forward pass applying $\tau_k^{\text{new}} = \tau_k + \alpha k_k + K_k(x_k^{\text{new}} - x_k)$ under a backtracking line search on α . Jacobians of f are computed by central finite differences. The regularizer μ is increased when $Q_{uu} + \mu$ fails a positive-definiteness check. The terminal weight P is taken as the diagonal matrix $\text{diag}(10, 40, 5, 20)$, shaped to match the infinite-horizon cost-to-go near the upright equilibrium. Torque limits $|\tau_k| \leq \tau_{\max}$ are enforced by clamping after each forward rollout.

4.2.3 Implementation Details

Execution is split into two phases. An offline solve runs once at construction over $N_{\text{off}} = 80$ steps (1.6 s) with up to 80 iterations, producing a nominal swing-up trajectory and time-varying feedback gains K_k^{off} . A multi-restart scheme with seven diverse seeds — zeros, $\pm$ constant pushes, bang-bang profiles tuned to the small-amplitude frequency $\omega = \sqrt{g/\ell_{c2}}$, and energy-pumping heuristics — guards against convergence to local minima that never reach upright.

At runtime, the controller tracks the offline plan via $\tau_k = \tau_k^{\text{off}} + K_k^{\text{off}}(x_k - x_k^{\text{off}})$. Once the plan is exhausted or the state error falls below a tolerance ϵ_{done} , a short-horizon receding iLQR takes over ($N_{\text{on}} = 35$, up to 3 iterations per solve, replanning every control cycle), warm-started by shifting the previous solution forward. Trajectory indices advance as $k = \lfloor t/\Delta t \rfloor$, making the controller rate-invariant with respect to the physics call rate. Weights $Q = \text{diag}(3, 15, 0.5, 2)$, $R = 0.5$ were obtained via an Optuna hyperparameter sweep evaluated across five robustness scenarios to avoid nominal-only overfitting.

4.3 Model Predictive Control (MPC)

4.3.1 Model

The MPC controller is based on the full nonlinear system dynamics introduced in Section 3. The system is represented in discrete-time form as

$$x_{k+1} = f(x_k, u_k),$$

where $f(\cdot)$ is obtained via numerical integration of the continuous-time dynamics.

4.3.2 Control Formulation

At each control step, MPC solves a finite-horizon optimal control problem of the form

$$\min_{u_{0:N-1}} \sum_{k=0}^{N-1} (x_k - x^*)^\top Q (x_k - x^*) + u_k^\top R u_k + (x_N - x^*)^\top P (x_N - x^*), \quad (7)$$

where the weighting matrices $Q \succeq 0$ and $R \succ 0$ follow the general formulation introduced in Section 2 and are tuned to balance tracking performance and control effort, subject to the system dynamics and input constraints

$$x_{k+1} = f(x_k, u_k), \quad |u_k| \leq u_{\max}. \quad (8)$$

Only the first control input u_0 is applied, and the optimization is repeated at the next time step following the receding horizon principle. The terminal cost matrix P is chosen as the solution to the discrete-time Riccati equation associated with the linearized system, providing a locally optimal infinite-horizon cost approximation and improving closed-loop stability.

4.3.3 Implementation Details

The OCP is formulated as a direct multiple-shooting problem and solved with CasADi [9] and IPOPT [10]. The system dynamics are integrated using fourth-order Runge–Kutta at $\Delta t = 0.04$ s over a horizon of $N = 20$ steps, giving a 0.8 s lookahead window. Torque limits are enforced as box constraints on the control variables within the NLP.

The terminal cost matrix P is computed from the DARE applied to the ZOH-discretized linearization at x^* , using the same Q and R as the stage cost. This provides the infinite-horizon LQR cost-to-go as a terminal penalty, improving closed-loop stability without extending the horizon.

At each control step the solver is warm-started by shifting the previous solution forward one step and pinning the initial state node to the current measurement via bounds. A control-rate penalty weighted by $S = 0.02$ is added to the stage cost to smooth the torque profile.

5 Simulation and Evaluation Setup

5.1 Simulation Environment

All controllers are evaluated using a shared simulation framework based on the MuJoCo physics engine [11]. The system dynamics are simulated at a fixed rate of 500 Hz, capturing the relevant system dynamics while keeping simulation overhead manageable.

To ensure a fair comparison of computational performance, the physics simulation and control computation are executed in separate processes. The simulation process advances the system state independently, while the controller operates on the most recent state provided via shared memory. This design prevents slower controllers from affecting the simulation rate, allowing computational cost to be measured independently of the physical system evolution.

A schematic overview of the simulation architecture is shown in Fig. 3.

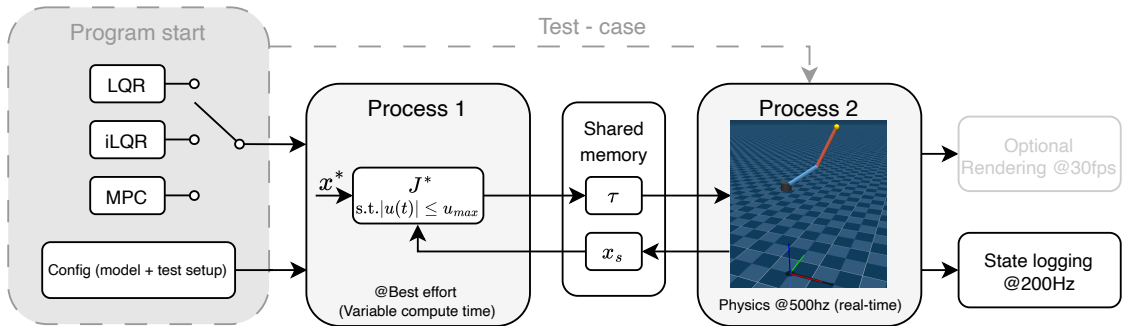


Figure 3: Simulation architecture illustrating the separation of physics and control processes, shared state communication, and logging pipeline.

5.2 Test Scenarios

All controllers are evaluated under a consistent set of test scenarios designed to assess nominal performance and robustness. These include nominal conditions, sensor noise, actuator limitations, model parameter mismatch, and external disturbances. In addition, combined stress scenarios are considered to evaluate performance under multiple simultaneous perturbations.

Each scenario is applied identically across all controllers, ensuring that performance differences arise solely from the control strategy.

5.3 Evaluation Metrics

Controller performance is evaluated using four metrics. Tracking accuracy is quantified by the RMS position error over the full episode. Control effort is measured by RMS torque. Settling time is defined as the first time instant at which both $|\tilde{\theta}_1|$ and $|\tilde{\theta}_2|$ remain within 7° of the upright equilibrium for a continuous window of 0.5s; a run is marked as FAIL if this condition is never met within the episode duration. Computational cost is reported as mean and maximum solve time per control step.

In addition to a desktop system (Apple MacBook Pro), computational performance was evaluated on a Raspberry Pi 3B+ (1.4 GHz quad-core, active cooling). CPU temperature was monitored and runs started only below 55°C to avoid thermal throttling.

6 Results

6.1 Linear Quadratic Regulator (LQR)

6.1.1 Nominal Performance

Figure 4 contrasts two configurations of the LQR controller: the full energy-shaped swing-up with catch-and-latch, and a pure LQR gain applied directly from the hanging position. The pure LQR case stabilizes the system, but only by saturating the actuator continuously for nearly two seconds, producing high-frequency torque chattering and converging to a spurious equilibrium far from the upright target. This is the expected consequence of operating a linear controller far outside its region of validity: the gain K , designed at $x^* = [\pi/2, 0, 0, 0]^\top$, applies large corrective torques that are inappropriate at the hanging configuration and cannot drive the system to the intended equilibrium without the nonlinear swing-up phase. The energy-shaped controller, by contrast, delivers the pendulum smoothly to the upright equilibrium with a clean torque profile and near-zero steady-state effort.

6.1.2 Catch Condition Sensitivity

Since the LQR gain is valid only near x^* , the switching condition determines the quality of the handoff between the nonlinear swing-up and linear stabilisation. Figure 5 characterises this dependence over a 10×10 grid of angle and velocity catch thresholds. The heatmap shows two distinct outcomes: black cells indicate failed stabilisation, while colored cells show the peak torque at the switching instant — which in the majority of successful cases reaches the actuator limit of $\pm 10\text{Nm}$, confirming that the energy pump consistently delivers the pendulum with high residual velocity.

Successful catches are confined to a narrow region requiring low angle thresholds and permissive velocity thresholds, reflecting the behaviour of the energy pump: the pendulum arrives at the top with high angular velocity, so the velocity threshold must be loose enough to allow a catch, while the angle threshold must remain tight to ensure the linear model is applied close to its linearisation point. The three representative traces illustrate qualitatively different outcomes. With well-tuned thresholds (green), the controller catches cleanly and stabilises within approximately 1s. With an overly tight velocity threshold (blue), the controller repeatedly reaches the upright region but refuses to latch, resulting in sustained oscillation. With a loose angle threshold (red), the controller catches while the state is still far from x^* , producing a large switching spike and ultimately converging to an incorrect equilibrium — a direct consequence of operating outside the linear region of attraction.

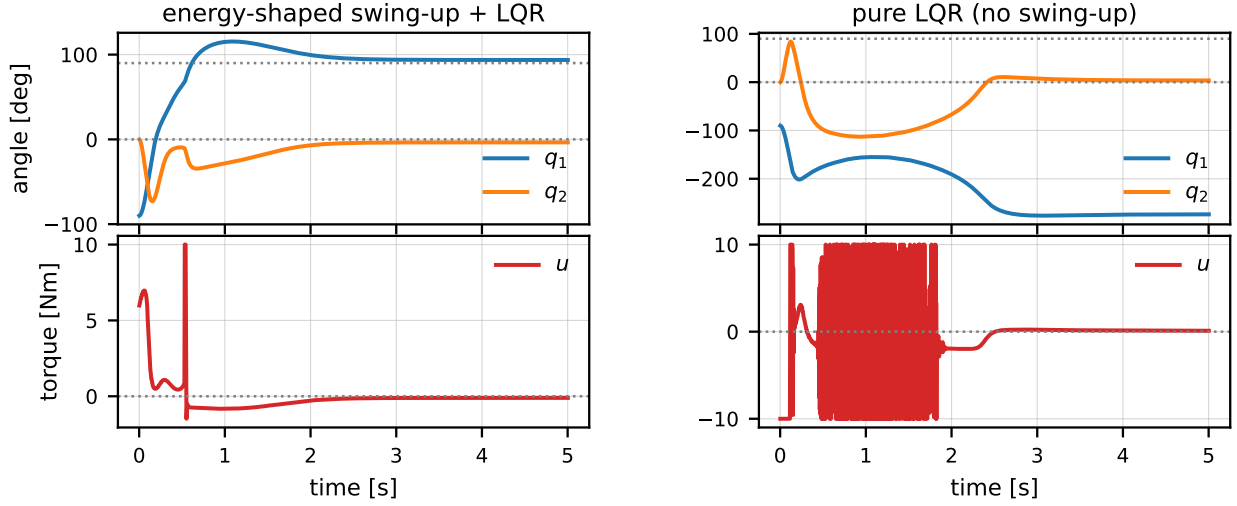


Figure 4: Closed-loop response of pure LQR (right) versus energy-shaped swing-up with LQR stabilisation (left). Without the swing-up phase, the linear gain saturates the actuator, produces high-frequency chattering, and fails to reach the upright equilibrium.

6.1.3 Noise Sensitivity

Figure 6 shows the effect of sensor noise on RMS torque and RMS position error. Torque increases by a factor of 7.6 between the nominal and high-noise cases, while position error increases by only 1.2 $\times$. This asymmetry is a direct consequence of the fixed-gain structure: the feedback law $u = -K\tilde{x}$ amplifies measurement noise linearly into the control signal with no filtering mechanism. Once the torque is predominantly noise-driven rather than state-corrective, further noise has limited additional effect on tracking performance, explaining the saturation visible in both curves at high noise levels. In practice, this sensitivity implies that LQR deployment on hardware requires either low-noise sensing or explicit state estimation prior to the feedback loop.

6.2 Iterative LQR (iLQR)

6.2.1 Nominal Performance

Figure 7 shows the closed-loop response of iLQR on the nominal scenario. The offline plan swings θ_1 from the hanging pose to within 0.1 rad of $\pi/2$ in 1.5 s. The torque profile shows the expected swing-up shape: a gradual ramp to roughly 3 N·m as the pendulum gains energy, followed by a brief bang-bang episode that saturates at ± 5 N·m near $t \approx 1.3$ s when θ_1 crosses the horizontal and θ_2 reverses. At that instant the gravitational and Coriolis loading on both links peaks simultaneously, and the clamp on $|\tau_k| \leq \tau_{\max}$ lets iLQR ride the constraint for a few steps before the commanded torque drops back to near zero as the pendulum crosses the unstable equilibrium. The offline-online switch at $t = 2.98$ s is visible as a small corrective pulse; the online phase thereafter holds θ_1 near $\pi/2$ with steady-state RMS torque below 0.5 N·m. Design parameters were obtained by an Optuna hyperparameter sweep of 1000 trials evaluated across five scenarios, so the cost weights reward robustness rather than nominal-only performance.

6.2.2 Robustness

The online feedback law applies the correction

$$\delta\tau_k = k_k + K_k \delta x_k, \quad (9)$$

where $\delta x_k = x_k - \bar{x}_k$ is the deviation from the nominal trajectory. When the measured state is corrupted by noise w_k this becomes

$$\delta\tau_k = k_k + K_k (\delta x_k + w_k), \quad (10)$$

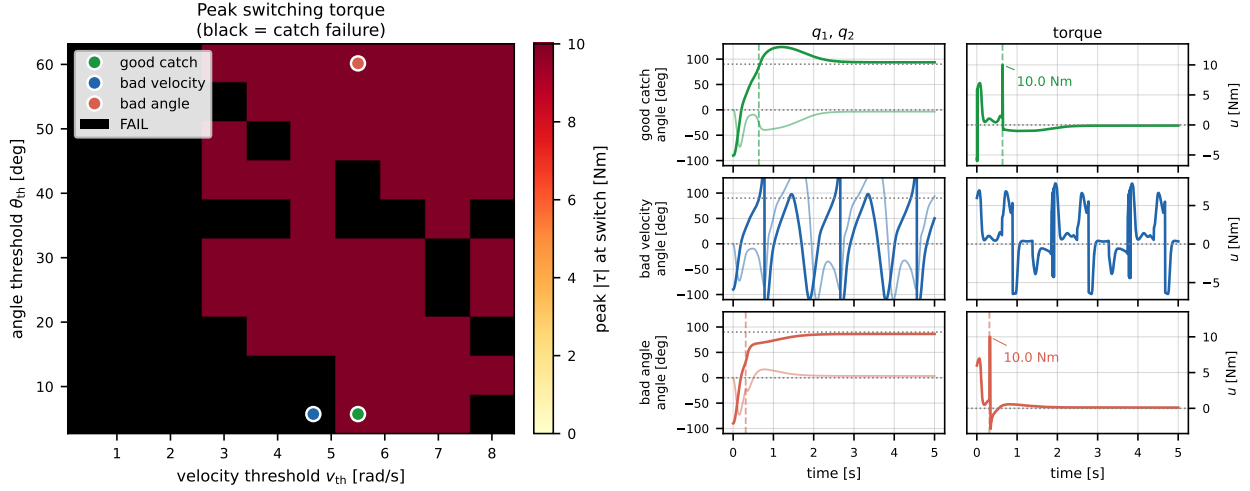


Figure 5: Left: peak switching torque over a grid of angle and velocity catch thresholds. Black cells indicate failed stabilisation; colored cells show the saturated switching torque. Right: representative traces for a well-tuned catch (green), a velocity threshold too tight to trigger (blue), and an angle threshold too loose, causing catch outside the linear region of attraction (red).

so measurement noise is amplified directly by the time-varying gains K_k with no smoothing mechanism. Figure 8 illustrates the consequence: cumulative control effort during the offline swing-up phase is identical between the nominal and high-noise runs, since the feedforward torques are precomputed and noise enters only through the feedback correction. The divergence is abrupt and persistent from the moment the online phase begins, confirming that the offline-online architecture localises the noise sensitivity entirely to the receding-horizon stage. RMS torque rises from $0.77 \text{ N}\cdot\text{m}$ in the nominal case to $2.70 \text{ N}\cdot\text{m}$ under high noise, while settling time degrades from 2.15 s to 3.47 s at mid noise and is lost entirely at the highest noise level.

Under model mismatch the offline feedforward torques become incorrect, and the short-horizon online replanner must absorb the discrepancy through feedback alone. The controller recovers at low and moderate mismatch levels but fails at the highest ($+30\%$ mass), where the accumulated feedforward error exceeds what the linear gains can correct. Torque constraints expose a structural limitation: the offline plan is computed at full $\pm 5 \text{ N}\cdot\text{m}$ authority, so restricting the available torque below the swing-up requirement causes the online law to clip its output without awareness of the constraint, failing to inject sufficient energy to reach the upright equilibrium. This is an architectural property rather than a tuning issue, and it explains why both torque-limited scenarios are failures despite the controller succeeding across most other perturbation types. Under combined stress the accumulated effect of noise, mismatch, and clipped authority causes the offline plan to break down entirely.

6.2.3 Computational Performance

Offline construction completes in under 3 s across all seven warm-start seeds and is paid once before the simulation clock starts. During online operation, finite-difference Jacobians dominate the inner loop with cost scaling as $\mathcal{O}(N_{\text{on}} \cdot n_x)$, giving highly predictable per-solve times. On the desktop, the online phase averages 4.6 ms with a worst case of 7.1 ms — both within the 20 ms control period. On the Raspberry Pi the mean rises to 35.3 ms and the worst case to 82.6 ms , exceeding one control period, yet iLQR succeeds in 6 of 13 scenarios on that platform because the latency compensator rolls the state estimate forward by $\lceil t_{\text{solve}}/\Delta t \rceil$ steps before each plan application. This bounded and predictable cost profile contrasts with MPC’s problem-dependent IPOPT iteration count, and is the primary reason iLQR retains real-time viability on constrained hardware where MPC does not (see Section 6.3.3).

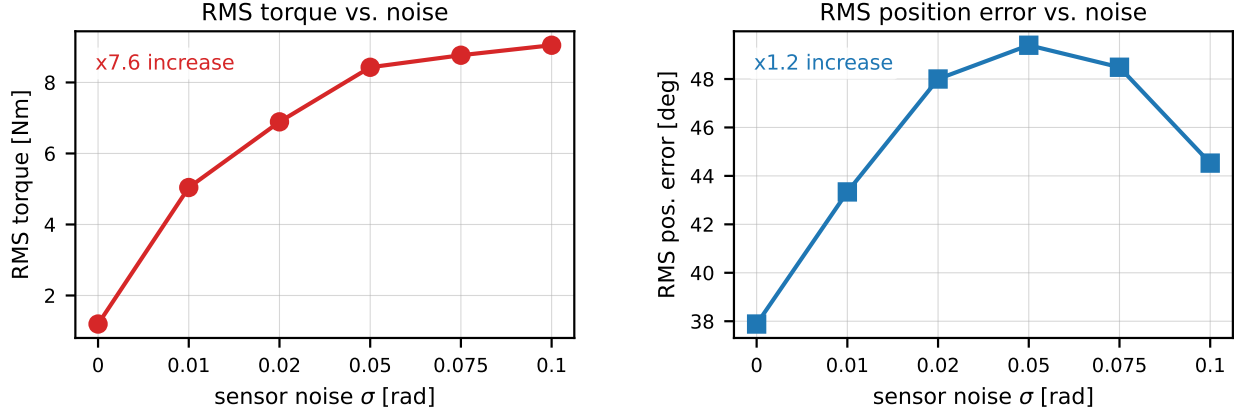


Figure 6: RMS torque (left) and RMS position error (right) as a function of sensor noise standard deviation σ . Torque grows 7.6 $\times$ while position error increases only 1.2 $\times$, reflecting direct amplification of measurement noise through the fixed feedback gain.

6.3 Model Predictive Control (MPC)

6.3.1 Nominal Performance

Figure 9 shows the nominal closed-loop response. Starting from the downward configuration, MPC drives the system to the upright equilibrium within approximately one second, with a torque profile consistent with an optimal swing-up: large initial control action to inject energy, tapering to near-zero as the pendulum approaches the target. A small residual steady-state deviation is visible, attributed to finite horizon length and numerical latency; it remains bounded, confirming that the DARE-based terminal cost provides an effective local approximation of the infinite-horizon value function. Moderate changes to Q and R had limited influence on closed-loop behavior, suggesting the feasible trajectory space is already strongly constrained by the nonlinear dynamics and input limits.

6.3.2 Robustness

Figure 10 shows performance under varying actuator limits. MPC remains effective over a wide range of admissible torques, with RMS error and settling time degrading gracefully as authority is reduced — a direct consequence of incorporating constraints explicitly in the optimization rather than through external saturation. A sharp feasibility threshold appears near $u_{\max} \approx 1$ Nm, below which the constrained NLP can no longer find a sufficient swing-up trajectory within the given horizon.

Under sensor noise, the control-rate penalty $S = 0.02$ in the stage cost acts as an implicit low-pass filter, producing smoother torque profiles than LQR or iLQR at the cost of slightly higher position error. This trade-off is visible in the comparative results of Section 6.4: MPC accepts marginally larger RMS position error in high-noise scenarios in exchange for significantly reduced actuator chatter. Model mismatch is handled well at moderate levels through the receding-horizon feedback, though performance degrades under the combined stress scenario where noise, mismatch, and torque limits interact simultaneously.

6.3.3 Computational Performance

Figure 11 characterises the central MPC trade-off: horizon length simultaneously determines control quality and computational cost. Below $N = 3$ the optimizer cannot generate a feasible swing-up trajectory. Performance improves with horizon up to a minimum settling time of 0.79s at $N = 6$ on the desktop, beyond which computational delay dominates and closed-loop performance degrades. Worst-case solve times grow faster than p95 values, amplifying timing variance — a particular concern for real-time control. The Raspberry Pi reaches

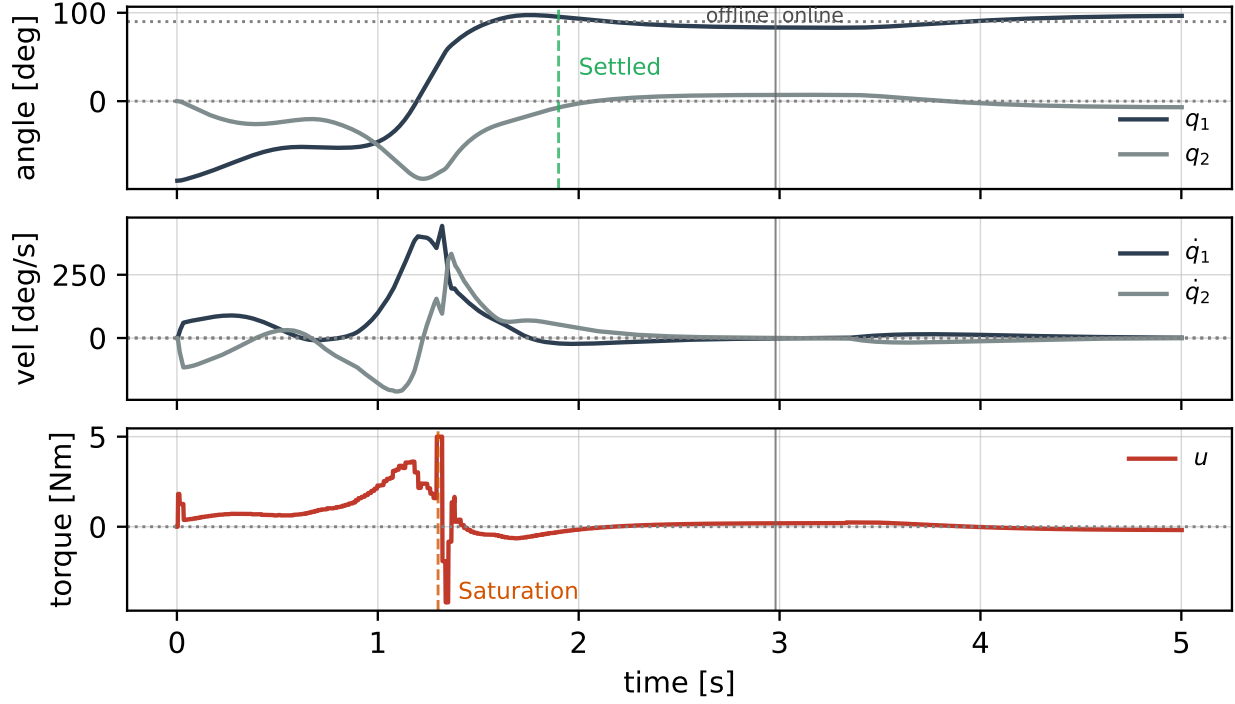


Figure 7: iLQR on the nominal scenario. Top: joint angles θ_1, θ_2 and their upright targets. Bottom: commanded torque τ with ± 5 N·m limits. The vertical line marks the offline/online phase switch at $t = 2.98$ s.

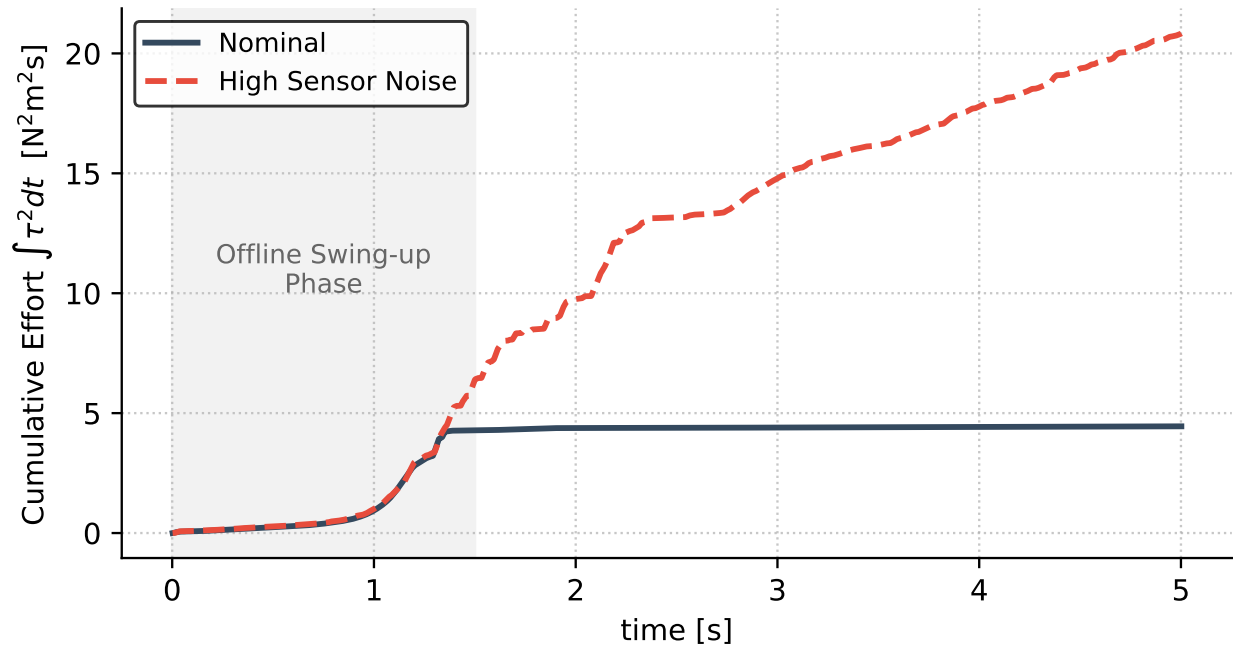


Figure 8: Cumulative control effort ($\int \tau^2 dt$) for the nominal and high sensor noise scenarios. The two curves are indistinguishable during the offline swing-up phase; divergence begins precisely at the offline–online switch, isolating the noise sensitivity to the receding-horizon stage.

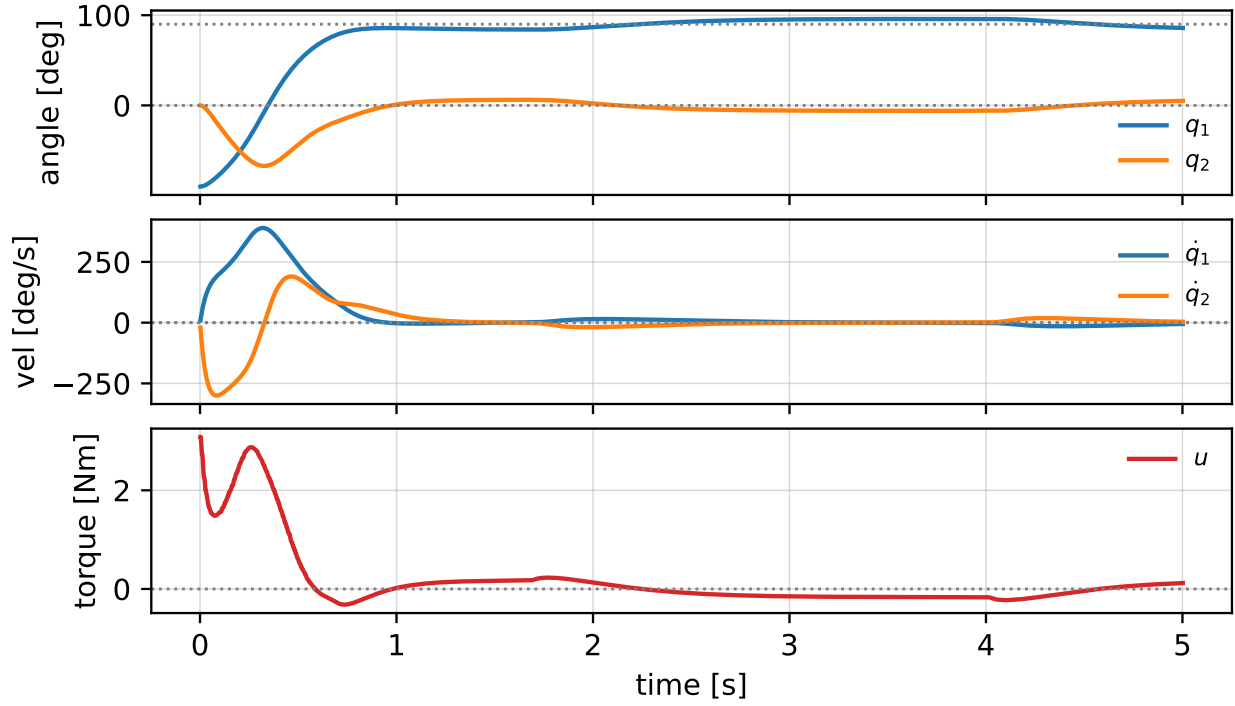


Figure 9: Nominal closed-loop response of nonlinear MPC. Top: joint angles. Middle: joint velocities. Bottom: control torque. Dotted lines indicate target values.

its feasibility limit near $N \approx 85$, compared to $N \approx 360$ on the desktop, confirming that horizon selection must be treated as a joint control-computation decision rather than a purely control-theoretic one.

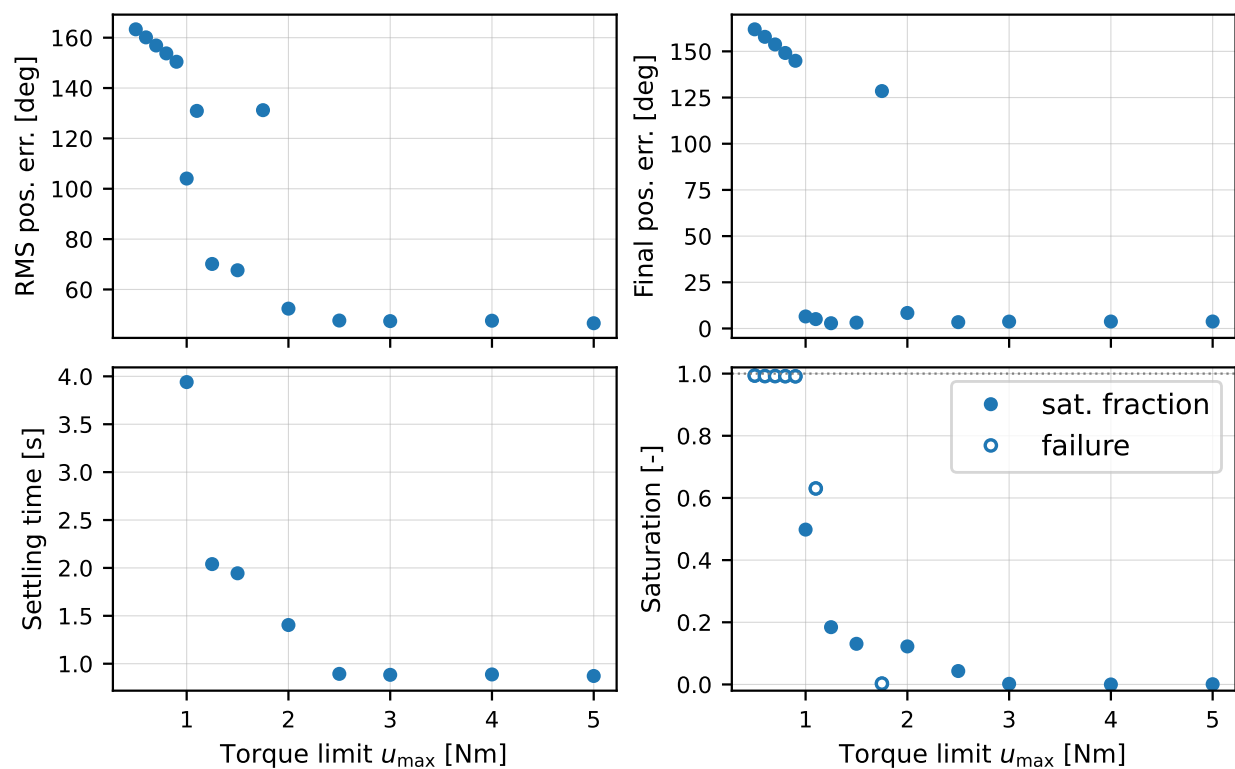


Figure 10: Effect of actuator torque limits on MPC performance. A sharp loss of stabilizability occurs near $u_{\max} \approx 1$ Nm. Filled markers denote successful runs; open markers denote failures.

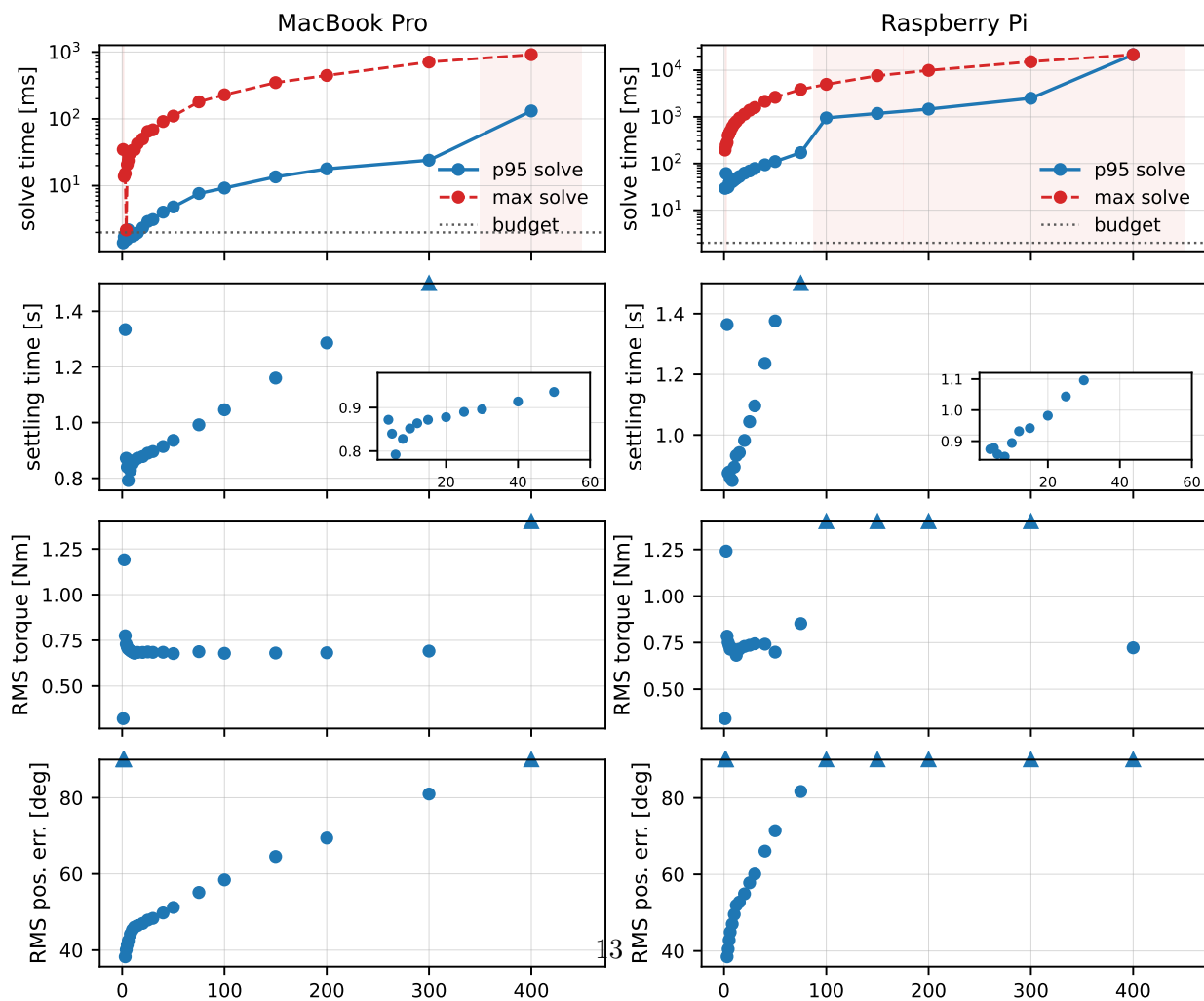


Table 1: Stabilisation success across all test scenarios (desktop). ✓ = settled; ✗ = failed.

	<i>Nominal</i>	<i>Noise low/mid</i>	<i>Noise high</i>	<i>Torque mid</i>	<i>Torque low</i>	<i>Mismatch low</i>	<i>Mismatch mid</i>	<i>Mismatch high</i>	<i>Impulse l/m/h</i>	<i>Combined</i>	<i>Total</i>
LQR	✓	✓	✗	✗	✗	✗	✗	✓	✓	✗	7/13
iLQR	✓	✓	✗	✗	✗	✓	✗	✗	✓	✗	7/13
MPC	✓	✓	✗	✓	✓	✓	✓	✗	✓	✗	10/13

6.4 Comparative Analysis

6.4.1 Stabilization Envelope

Figure 12 and Table 1 summarise performance across all 13 scenarios. MPC succeeds in 10 of 13 cases; LQR and iLQR each succeed in 7, but with structurally different failure profiles. LQR and iLQR both fail every torque-limited scenario: LQR clips its output without awareness of the constraint; iLQR’s offline plan was computed at full authority and the online feedback cannot recover from insufficient energy injection. MPC succeeds in both torque-limited cases ($u_{\max} \in \{1, 2\}$ Nm) because the constraint is enforced inside the NLP at every step. Under model mismatch, LQR’s failures originate in the energy-pump catch condition rather than the linear gain itself, while iLQR degrades as the offline plan diverges from the perturbed plant. MPC fails only at the highest mismatch level (+30% mass). All three controllers fail under combined stress and high sensor noise, confirming that none provides unconditional robustness within the tested parameter range.

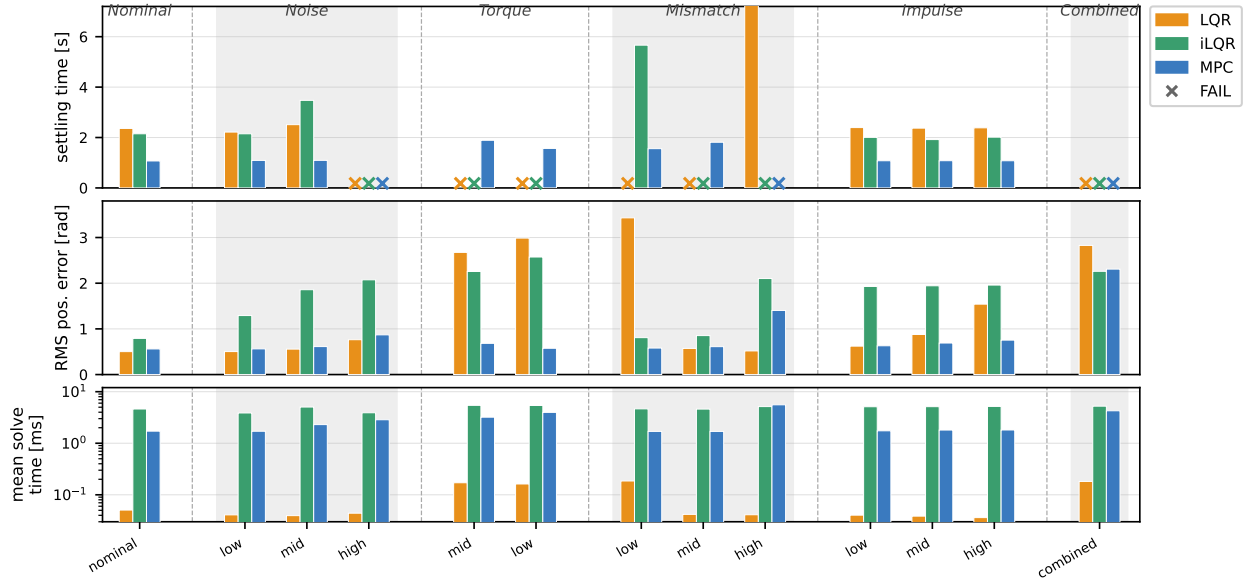


Figure 12: Performance comparison across all 13 test scenarios. Top: settling time (× marks failures). Middle: RMS position error. Bottom: mean solve time (log scale). Alternating bands indicate scenario groups.

6.4.2 Control Quality Under Perturbation

Under nominal conditions all three controllers achieve comparable RMS position errors (0.50, 0.79, 0.56 rad for LQR, iLQR, and MPC). Differences emerge under perturbation. At the highest noise level, LQR produces 9.2 Nm RMS torque — saturating the actuator near-continuously — while MPC produces 3.4 Nm at the same

noise level. The control-rate penalty in the MPC stage cost acts as an implicit low-pass filter, trading a modest increase in position error (0.87 vs. 0.76 rad) for substantially smoother actuation. iLQR falls between the two at 2.7 Nm RMS torque but with higher position error (2.1 rad) than either, as its time-varying feedback gains amplify noise without any rate-smoothing mechanism. Impulse disturbances are handled well by all controllers that stabilise, with settling times within 0.3 s of nominal, indicating a sufficient basin of attraction for the tested magnitudes.

6.4.3 Computational Feasibility and Embedded Deployment

Figure 13 compares mean and worst-case solve times on both platforms. The three-order-of-magnitude spread on a logarithmic axis captures the essential trade-off. LQR’s matrix-vector multiply takes 0.05 ms on the desktop and 0.36 ms on the Pi — trivially fast on both. iLQR’s online phase averages 4.6 ms on the desktop with a bounded worst case of 7.1 ms, both within the 20 ms control period. On the Pi the mean rises to 35.3 ms, exceeding one control period, yet iLQR still succeeds in 6/13 scenarios because the latency compensator rolls the state estimate forward before each plan application. MPC presents a qualitatively different profile: nominal mean of 1.7 ms on the desktop, but worst-case spikes of 44 ms already exceed the budget on a single step, and on the Pi the worst case reaches 621 ms — a $21\times$ slowdown compared to the $7\times$ experienced by LQR and iLQR. This variance reflects the problem-dependent iteration count of IPOPT and makes MPC fundamentally difficult to schedule under hard real-time constraints, despite its superior robustness.

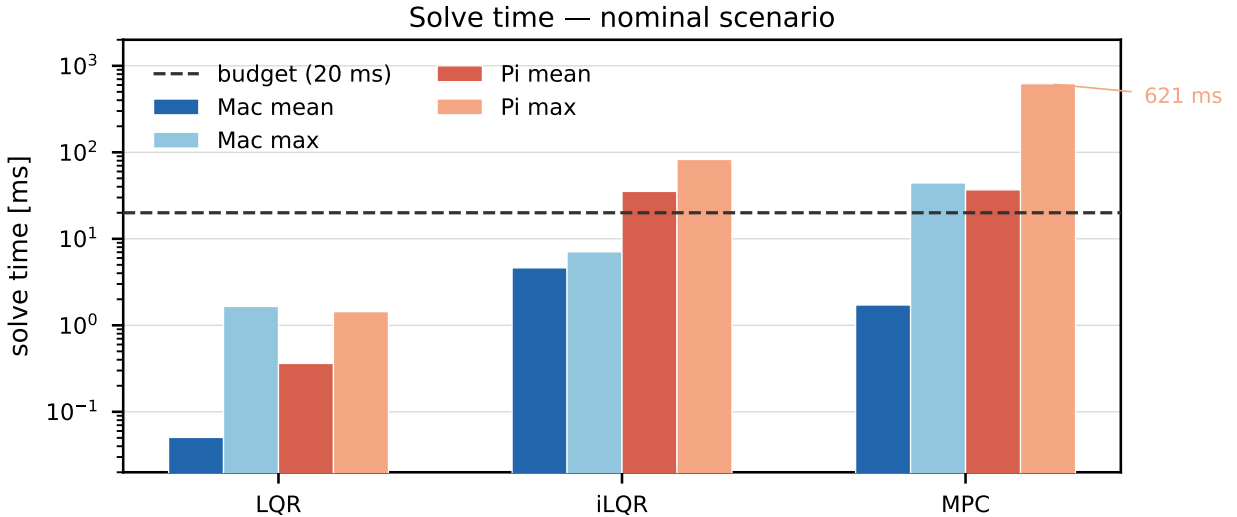


Figure 13: Mean and worst-case solve times for all three controllers on desktop and Raspberry Pi (nominal scenario). The dashed line marks the 20 ms control period. MPC’s worst-case Pi latency of 621 ms reflects the unpredictable iteration count of the interior-point solver under constrained hardware.

Taken together, the results support a clear selection criterion. When actuator constraints must be respected online and computational resources are sufficient, MPC is the appropriate choice. When the system must run on embedded hardware with hard real-time guarantees, iLQR’s offline-online architecture delivers full nonlinear capability at a predictable and bounded cost. LQR remains valuable wherever the system operates close to equilibrium with low-noise sensing — its computational overhead is negligible on any platform considered here.

7 Discussion

7.1 What the Results Reveal About Controller Design

The three controllers tested here represent successive relaxations of a single limiting assumption. LQR assumes linear dynamics and no constraints; iLQR relaxes linearity through iterative trajectory optimization; MPC additionally incorporates explicit constraint handling at each step. The results confirm that each relaxation improves robustness at a computational cost — but the relationship is not linear, and the failure modes are structurally different in ways that matter for deployment.

The most revealing result is that LQR and iLQR achieve the same success count (7/13) despite fundamentally different architectures. LQR fails because its basin of attraction is too small and its gain amplifies disturbances directly. iLQR fails for a different reason: its online phase is constraint-blind, so any scenario that reduces available torque below the offline plan’s assumption is unrecoverable. This distinction is practically important — iLQR’s failure mode is architectural rather than a matter of tuning, and cannot be resolved without modifying the online controller structure.

The noise results expose a property of fixed-gain controllers that is easy to overlook: the feedback gain K amplifies measurement noise with no mechanism to smooth the output. The $7.6\times$ torque increase observed under high noise for LQR, and the analogous behavior in iLQR, is a direct consequence of operating a linear proportional controller on a noisy signal. MPC’s implicit low-pass behavior through the control-rate penalty is not a deliberate filter design — it emerges from the cost structure — and represents one of the less obvious practical advantages of the receding-horizon formulation.

7.2 Limitations

The swing-up wrapper around LQR introduces a design degree of freedom — the catch condition thresholds — that is not part of the LQR formulation itself and requires separate tuning. The 2D sweep in Section 6.1 shows that the success region in threshold space is narrow, making the system sensitive to this choice. A principled approach would use a Lyapunov-based switching condition derived from the LQR value function rather than hand-tuned geometric thresholds.

For iLQR, the offline trajectory is computed for nominal parameters. No mechanism exists to update it online under mismatch, which limits the controller’s robustness to plant changes in a way that MPC does not share. The Optuna hyperparameter sweep used for tuning partially compensates by optimizing across multiple scenarios, but this is a heuristic rather than a guarantee.

MPC’s terminal cost is computed from the DARE at the linearized equilibrium, which is the theoretically correct choice for local stability guarantees. However, the finite horizon means the controller cannot plan a globally optimal swing-up — it discovers the trajectory through receding-horizon iteration. The minimum feasible horizon of $N = 3$ observed experimentally reflects the shortest window in which the optimizer can find a path to the upright configuration, not a design choice.

7.3 Implications for Practice

The computational results on the Raspberry Pi provide a concrete answer to the embedded deployment question. LQR is feasible on any platform — its 0.36 ms Pi solve time represents no constraint. iLQR’s mean of 35 ms exceeds the control period but the latency compensator preserves functionality at a cost of increased settling time. MPC’s worst-case of 621 ms makes hard real-time scheduling impossible without significant horizon reduction, which in turn degrades control quality. For a system with a 20 ms control period, iLQR is the only method among the three that delivers full nonlinear swing-up capability within an acceptable computational envelope on constrained hardware.

8 Conclusion

This study compared LQR, iLQR, and nonlinear MPC on a rotary double pendulum across 13 test scenarios covering sensor noise, actuator limits, model mismatch, and impulse disturbances. The quantitative findings establish clear operational boundaries. LQR is computationally trivial (0.05 ms per step) but fails in 6 of 13 scenarios due to its restricted basin of attraction and direct noise amplification through the fixed gain. iLQR matches LQR’s success count (7/13) through a different mechanism — reliable nonlinear swing-up via offline trajectory optimization — but shares the vulnerability of fixed-gain feedback online and fails completely under torque constraints it cannot anticipate. MPC succeeds in 10 of 13 scenarios by incorporating constraints and a smoothing cost directly into the optimization, but at the expense of unbounded worst-case solve times that reach 621 ms on embedded hardware.

The central finding is that the performance hierarchy depends critically on the deployment context. On a desktop system with ample compute, MPC is the superior choice. On resource-constrained hardware with hard real-time requirements, iLQR’s offline-online architecture provides the best achievable trade-off between nonlinear capability and computational predictability. The results also highlight that equal success counts do not imply interchangeable controllers — the failure modes of LQR and iLQR are structurally distinct and respond differently to the same disturbances, a distinction that only becomes visible through systematic cross-scenario evaluation of the kind performed here.

References

- [1] J. Pratt, J. Carff, S. Drakunov, and A. Goswami, “Capture point: A step toward humanoid push recovery,” in *Proc. IEEE-RAS Int. Conf. Humanoid Robots*, pp. 200–207, 2006.
- [2] A. De Luca and G. Oriolo, “Underactuated manipulators: Control properties and techniques,” *Machine Intelligence and Robotic Control*, vol. 5, no. 3, pp. 72–80, 2003.
- [3] R. Tedrake, “Underactuated robotics: Algorithms for walking, running, swimming, flying, and manipulation,” 2024. Course Notes for MIT 6.832.
- [4] W. Li and E. Todorov, “Iterative linear quadratic regulator design for nonlinear biological movement systems,” in *Proc. 1st Int. Conf. Informatics in Control, Automation and Robotics*, pp. 222–229, 2004.
- [5] J. B. Rawlings and D. Q. Mayne, *Model Predictive Control: Theory and Design*. Nob Hill Publishing, 2009.
- [6] Y. Tassa, N. Mansard, and E. Todorov, “Control-limited differential dynamic programming,” in *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1168–1175, 2014.
- [7] D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert, “Constrained model predictive control: Stability and optimality,” *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.
- [8] Y. Wang and S. Boyd, “Fast model predictive control using online optimization,” *IEEE Transactions on Control Systems Technology*, vol. 18, no. 2, pp. 267–278, 2010.
- [9] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, “Casadi: A software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [10] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, no. 1, pp. 25–57, 2006.
- [11] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems*, pp. 5026–5033, IEEE, 2012.